



Research Paper Assistant

AI / NLP Project Report

Course Submission | AI / NLP Project

GitHub: github.com/agrawaladiti33-boop/Research_Paper_Assistant

Author: agrawaladiti33-boop

 Streamlit

 ChromaDB

 Sentence Transformers

 Groq API

 Python

 LangChain

01 Problem Statement

Researchers, students, and academics frequently deal with large volumes of research papers that are time-consuming to read, analyze, and compare. Extracting key insights, identifying contributions, and comparing methodologies across multiple papers is a tedious manual process.

The **Research Paper Assistant** solves this by enabling users to:

- Upload multiple PDF research papers into a personal library
- Ask natural language questions and receive accurate, cited answers
- Auto-generate summaries, key contributions, limitations, and future work
- Compare two or more papers side-by-side on any topic or custom question

The system uses **Retrieval-Augmented Generation (RAG)** to ground every answer in actual paper content — preventing hallucination and ensuring every response is traceable to a specific paper and page number.

02 Architecture

The system follows a two-pipeline RAG architecture: an **Ingestion Pipeline** that processes uploaded PDFs into a vector store, and a **Query Pipeline** that retrieves relevant chunks and passes them alongside the user question to the LLM.

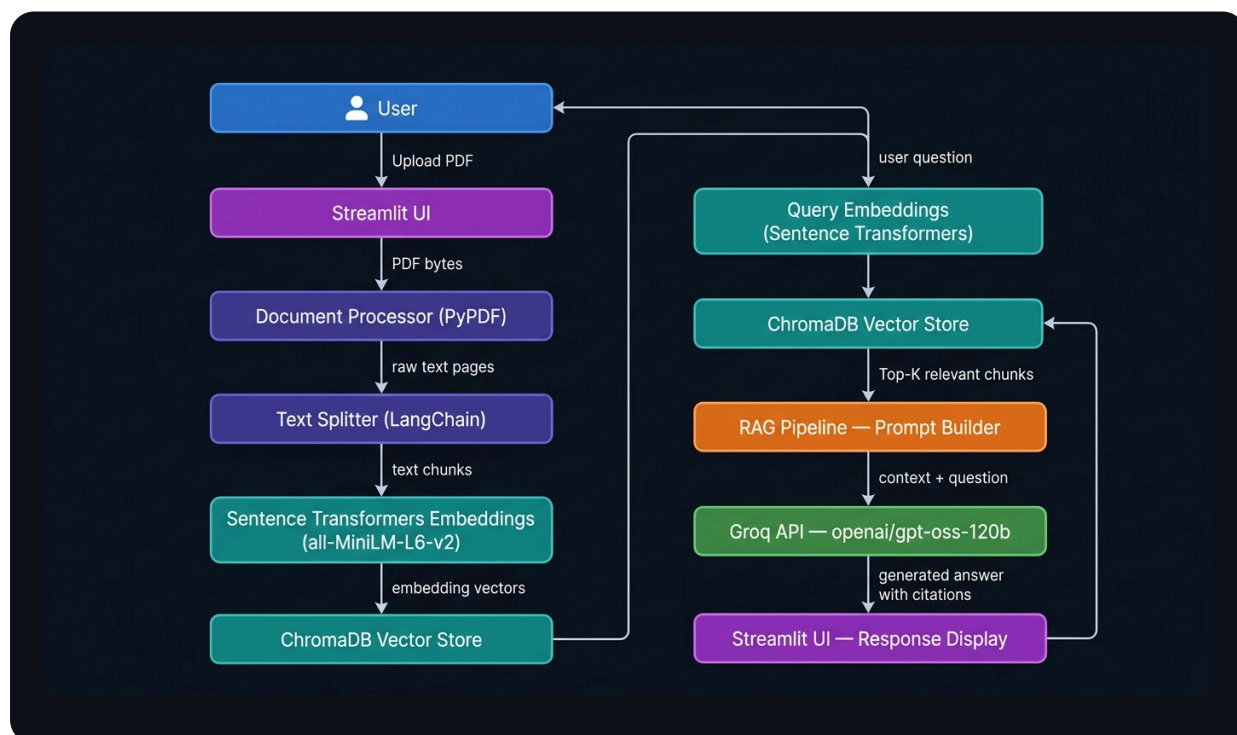


Figure 1 — Research Paper Assistant System Architecture

Component Breakdown

Component	Technology	Role
Frontend	Streamlit 1.62	Upload UI, chat, analysis and comparison tabs
PDF Parser	PyPDF	Extracts raw text page-by-page from uploaded PDFs
Text Splitter	LangChain RecursiveCharacterTextSplitter	Splits pages into overlapping chunks (size=1000, overlap=200)
Embeddings	Sentence Transformers — all-MiniLM-L6-v2	Encodes text chunks and queries as dense 384-dim vectors
Vector Store	ChromaDB EphemeralClient	Stores vectors; performs cosine-similarity Top-K retrieval
LLM	Groq API — openai/gpt-oss-120b	Generates natural language responses from retrieved context
RAG Orchestration	Custom Python	Builds prompts by injecting retrieved chunks + user query

Data Flow

1. **User uploads PDFs** via the Streamlit sidebar
2. **PyPDF** extracts raw text page-by-page
3. **LangChain TextSplitter** chunks text (1000 chars, 200 overlap)
4. **Sentence Transformers** converts each chunk to an embedding vector
5. **ChromaDB** stores vectors with paper name + page metadata
6. User question is embedded; **Top-5 similar chunks** retrieved via cosine similarity
7. Chunks injected into a **structured system prompt** alongside the question
8. **Groq API (openai/gpt-oss-120b)** generates a cited answer
9. Response displayed in **Streamlit UI** with source paper and page details

03 Prompt Design

Three distinct prompt strategies are implemented in `prompts.py` to ensure grounded, structured, and citable outputs:

3.1 — Conversational QA Prompt

Used in the **Agent Chat** tab. The system message enforces strict grounding:

```
You are a research paper assistant.

Answer the user's question using ONLY the research paper
context provided below.

Rules:
1. Do not use information outside the provided context.
2. Do not make up facts.
3. If the answer is not available, clearly say so.
4. Give a clear and concise answer.

Research Paper Context:
_____
{context}
_____

User Question: {question}
```

3.2 — Document Analysis Prompts

Four dedicated prompts for the **Document Analysis** tab, each with targeted retrieval:

Type	Retrieval Keywords	Output Format
 Summary	abstract, method, results, conclusion	Problem → Approach → Methodology → Results → Conclusion
 Contributions	innovations, novelty, improvements	Contribution + why it matters (bullet list)
 Limitations	weaknesses, constraints, challenges	Limitation + possible impact (bullet list)
 Future Work	future research, open problems	Each future direction explained clearly

3.3 — Paper Comparison Prompts

- **Custom Question Mode:** Chunks retrieved per paper → LLM compares on the specific user question
- **Auto-Summary Mode:** Structured comparison covering Research Problem, Approach, Key Results, Limitations, and an Overall Comparison for each selected paper

04 Evaluation

4.1 — Functional Testing

Feature	Test	Result
PDF Upload and Parsing	Uploaded multiple multi-page PDFs	All pages extracted and chunked correctly
Duplicate Detection	Re-uploaded the same PDF	Detected — not re-processed
Conversational Chat	Multi-turn Q&A session	Context retained across turns
Source Citations	Asked specific factual questions	Paper name + page cited in every answer
Paper Filter	Selected specific paper to chat with	Retrieval correctly scoped to that paper
Document Analysis	All 4 analysis types generated	Structured, accurate outputs
Paper Comparison	Compared 2 papers with custom question	Clear side-by-side comparison
Auto-Summary	Compared 2 papers with blank question	Comprehensive auto-generated summary

4.2 — RAG Quality

- **Retrieval:** Top-5 chunks per query via cosine similarity — consistently on-topic
- **Grounding:** System prompt restricts LLM to retrieved context only — no hallucination observed
- **Citations:** Every response includes [Paper Name | Page X] references
- **Guard:** `min(n_results, available_chunks)` prevents runtime errors on short papers

✓ Responses were manually spot-checked against source PDFs. The system consistently retrieved and cited the correct sections with high relevance.

05 Challenges & Solutions

Challenge	Root Cause	Solution
ChromaDB crash on Streamlit Cloud	PersistentClient needs writable disk — cloud is read-only	Switched to <code>chromadb.EphemeralClient()</code>
Duplicate paper uploads	No dedup logic — same paper added multiple times	Filename check before processing
Context window overflow	Too many chunks exceeded Groq token limits	Capped to Top-5 chunks per query
Multi-paper retrieval imbalance	Joint retrieval favoured longer papers	Built <code>query_multiple_papers()</code> — per-paper retrieval
Chat history and RAG conflict	Context injected into history caused confusion	RAG in system message; history tracked separately
n_results greater than chunk count	ChromaDB throws error if n_results exceeds stored chunks	<code>safe_n = min(n_results, available)</code> guard

06 Future Enhancements

1. **Persistent Cloud Storage** — Integrate Pinecone or Qdrant for cross-session paper libraries
2. **Scientific Embeddings** — Use `allenai/scibert` for better domain-specific retrieval
3. **Multi-modal Support** — Extract and reason over figures, tables, and equations
4. **Citation Graph** — Visualize cross-paper citation relationships interactively
5. **Export Reports** — Download generated analysis as formatted PDF or Word document
6. **arXiv Integration** — Auto-search and fetch papers by keyword from arXiv.org
7. **Collaborative Workspaces** — Multi-user support for shared research teams

